ADR 0005: Images and CDN

Status	Accepted for the MVP (Cloudinary direct); target storage proposed
Date	2026-10-19
Related	`03-system-architecture.md` sections 3 and 9, `06-mvp-slice-and-migration.md` sections 2 and 9

Context

Most catalog images are hot-linked from brand servers (Zara, M&S, Bonkers and others). Only vendor uploads and one import path are stored on Cloudinary. The app wraps **every** image address in `/v1/items/getimage?url=...`, so each photo goes through the Django server (a development server in production), with no timeout, no cache headers, and an open fetch of any URL. Measured on production: about 0.9 to 1.1 seconds through the proxy against 0.08 to 0.6 seconds direct, with Cloudflare reporting `DYNAMIC` (not cached). Cloudinary already has its own CDN, but the app never uses it directly.

Options

1. **Keep the proxy and cache it** (allow-list, timeout, cache headers, a Cloudflare cache rule). A stop-gap only.
2. **Cloudinary for everything:** vendor uploads and imported images stored there; the app loads Cloudinary URLs directly; resizing by URL. Already integrated.
3. **Cloudflare R2 (S3 compatible) with the Cloudflare CDN:** cheap egress, presigned uploads, you build the resizing step. This is what the scaffold uses (`integrations/r2.py`).
4. **S3 and CloudFront:** standard but more setup for no gain here.

Decision

- **MVP:** Option 2 for vendor products. Vendors upload three or more photos straight to Cloudinary; the app shows Cloudinary URLs directly (a helper that skips the proxy for Cloudinary hosts). The proxy is hardened (allow-list of brand hosts, 5 second timeout, size cap, correct content type, long cache headers) and cached by a Cloudflare rule only for scraped demo images.
- **Target:** Option 3. Presigned uploads to object storage, three generated sizes (thumbnail, card, full) in a modern format, served from a CDN domain such as `images.shazlo.store`. The app never calls the proxy.
- Scraped brand images stay **link-out only**; copying or hot-linking brand photos needs the brands' permission, which is part of the business-model decision.

Consequences

- Vendor photos stop going through the API server.
- Two storage systems exist until Cloudinary is retired; `product_images.key` stores a URL or an object key so both work.
- Cost on Cloudinary grows with traffic; revisit when the bill matters.

Revisit when

Image traffic cost becomes significant, or the team builds the resizing pipeline for object storage.